Worked for 48s

Yes. For your **200K-loan dataset**, the Silver layer should not just be a copy of Raw. The goal is to make the data **clean, typed, consistent, validated, and analytics-ready**, while keeping the business meaning intact.

I inspected the uploaded `loans_200k.csv`: it has **96 columns**, including UUIDs, financial amounts, timestamps, Boolean flags, categorical fields, and several JSON/JSONB-style columns. That makes it a good candidate for a structured Silver transformation.

Recommended Raw → Silver flow

```
PostgreSQL
    ↓
Databricks Auto Loader / Connector
    ↓
RAW / BRONZE
    ↓
┌──────────────────────────────┐
│       DATA WRANGLING         │
│                              │
│  1. Schema enforcement       │
│  2. Type casting             │
│  3. Null handling            │
│  4. Duplicate removal        │
│  5. Data-quality validation  │
│  6. Standardization          │
│  7. Business-rule checks     │
│  8. Derived columns          │
│  9. JSON normalization       │
│ 10. Audit columns            │
└──────────────────────────────┘
    ↓
SILVER
    ↓
Gold / Business Layer
```

1. Enforce proper data types

Your CSV/Raw ingestion currently has many fields as `string/object`.

For Silver, explicitly cast them.

IDs

```
id                → string/binary UUID
user_id           → string/binary UUID
application_number → string
```

Don't perform calculations on UUIDs.

Financial columns

These should be `DECIMAL`, not floating point:

```
amount
fee
fee_percentage
total_payable
interest_rate
interest_rate_after_due_date
total_amount_paid
total_penalty_amount
bounce_amount
waiver_amount
refund_amount
emi_amount
```

For example:

```sql
CAST(amount AS DECIMAL(18,2))
```

For percentages:

```sql
CAST(interest_rate AS DECIMAL(5,2))
```

This is particularly important for financial analytics.

---

2. Convert all date/time columns

Your dataset contains:

```
created_at
updated_at
disbursed_at
due_date
closed_at
rejected_at
marked_npa_at
last_call_date
```

Convert them from strings to:

```
TIMESTAMP
```

For example:

```sql
CAST(created_at AS TIMESTAMP) AS created_at
```

Since your generated data uses UTC timestamps, keep the timezone handling consistent throughout the pipeline.

---

3. Clean the `tenure` column

Currently:

```
3 Months
6 Months
12 Months
24 Months
36 Months
```

Keep the original field if useful, but create:

```
tenure_months
```

Example:

```
"12 Months" → 12
"24 Months" → 24
```

In PySpark:

```python
from pyspark.sql.functions import regexp_extract, col

df = df.withColumn(
    "tenure_months",
    regexp_extract(col("tenure"), r"(\d+)", 1).cast("int")
)
```

This will be much easier for analytics.

---

4. Standardize categorical columns

You have many categorical columns:

```
status
payment_status
loan_type
reason
collection_status
after_disbursal_status
overdue_substatus
category
source
applied_via
platform
policy
bureau_result
bsa_result
```

Check for:

```
Leading/trailing spaces
Different capitalization
Unexpected values
NULL
```

For example:

Python Run

```python
from pyspark.sql.functions import trim, col

df = df.withColumn(
    "status",
    trim(col("status"))
)
```

You want one consistent representation:

```
Disbursed
Approved
Rejected
Closed
```

rather than:

```
disbursed
DISBURSED
```

```
Disbursed
```

---

5. Remove duplicate loans

Your natural business key is:

```
loan_id / id
```

You have `loan_seq` in PostgreSQL as your technical ingestion key.

In Silver, I would retain both:

```
loan_seq → technical ingestion key
id       → loan business identifier
```

Check duplicates:

SQL

```sql
SELECT
    id,
    COUNT(*) AS cnt
FROM raw_loans
GROUP BY id
HAVING COUNT(*) > 1;
```

For your Silver table:

Python Run

```python
df = df.dropDuplicates(["id"])
```

For incremental pipelines, however, don't blindly use `dropDuplicates()` on the entire streaming dataset. Use a proper **MERGE/upsert strategy** if records can be updated.

---

6. Handle NULL values correctly

Don't replace every NULL with zero.

For example:

```
refund_amount = NULL
```

could mean "not applicable/not recorded."

But:

```
total_amount_paid = NULL
```

might need to become `0` depending on your business definition.

Good candidates for zero

Potentially:

```
total_amount_paid
total_penalty_amount
bounce_amount
waiver_amount
refund_amount
```

But only after establishing the business rule.

Don't do this blindly

Python Run

```python
df.fillna(0)
```

That would destroy the meaning of NULLs in fields such as:

```
disbursed_at
closed_at
rejected_at
collection_agency
rejected_by
```

---

7. Validate status/date consistency

This is a very important Silver-layer step.

Your data should follow rules such as:

Disbursed loan

```
status = Disbursed
        ↓
disbursed_at IS NOT NULL
```

Rejected loan

```
status = Rejected
        ↓
rejected_at IS NOT NULL
```

Closed loan

```
status = Closed
        ↓
closed_at IS NOT NULL
disbursed_at IS NOT NULL
```

You can create data-quality flags:

```sql
CASE
    WHEN status = 'Disbursed'
         AND disbursed_at IS NULL
    THEN false

    WHEN status = 'Rejected'
         AND rejected_at IS NULL
    THEN false

    ELSE true
END AS is_valid_status_timeline
```

---

8. Validate financial logic

This is where your project becomes much more realistic from a Data Engineering perspective.

Rule 1

```
amount >= 0
```

Rule 2

```
fee >= 0
```

Rule 3

```
total_amount_paid >= 0
```

Rule 4

```
total_amount_paid <= total_payable
```

may not always hold if penalties are included separately, so define the business rule carefully.

Rule 5

```
refund_amount <= total_amount_paid
```

where applicable.

Rule 6

```
emi_amount > 0
```

for disbursed loans.

---

9. Create useful financial derived columns

This is one of the biggest improvements I'd make.

Outstanding amount

SQL

```sql
GREATEST(
    total_payable
    - COALESCE(total_amount_paid, 0),
    0
) AS outstanding_amount
```

Net paid amount

SQL

```sql
COALESCE(total_amount_paid, 0)
- COALESCE(refund_amount, 0)
AS net_amount_paid
```

Total adjustments

```sql
COALESCE(waiver_amount, 0)
+
COALESCE(refund_amount, 0)
AS total_adjustments
```

Net customer value

```sql
COALESCE(total_amount_paid, 0)
- COALESCE(refund_amount, 0)
AS gross_customer_value
```

You can then use these fields in your Gold/LTV analysis.

---

10. Create loan age

Very useful for portfolio analysis.

```sql
DATEDIFF(
    CURRENT_DATE(),
    DATE(created_at)
) AS loan_age_days
```

Or based on disbursement:

```sql
DATEDIFF(
    CURRENT_DATE(),
    DATE(disbursed_at)
) AS days_since_disbursement
```

For closed loans, you can instead calculate:

```
closed_at - disbursed_at
```

---

11. Create delinquency indicators

Your dataset already has:

```
overdue_substatus
```

Turn this into analytical flags.

For example:

```sql
CASE
    WHEN overdue_substatus = 'No Overdue'
    THEN 0
    ELSE 1
END AS is_overdue
```

Then:

```sql
CASE
    WHEN overdue_substatus IN ('61-90 DPD', '90+ DPD')
    THEN 1
    ELSE 0
END AS is_high_dpd
```

And:

```sql
CASE
    WHEN overdue_substatus = '90+ DPD'
    THEN 1
    ELSE 0
END AS is_90_plus_dpd
```

---

12. Create NPA flag

You already have:

```
marked_npa_at
```

Create:

SQL

```sql
CASE
    WHEN marked_npa_at IS NOT NULL
    THEN TRUE
    ELSE FALSE
END AS is_npa
```

This makes downstream analysis much easier.

---

13. Standardize Boolean columns

You already have many Boolean fields:

```
paid
is_risky_customer
is_edited
is_in_lms
is_settlement
is_audit_done
is_sent_in_mis
is_waivered
is_review_done
is_foreclosed
is_bsa_manual
auto_disbursal_checks_passed
```

Make sure they're actually Boolean in Silver:

```
TRUE
FALSE
```

rather than:

```
"true"
"false"
"True"
"False"
```

---

14. Validate Boolean/business logic

This is another good DQ rule.

For example:

```
is_foreclosed = TRUE
        ↓
sub_status should = Foreclosed
```

Similarly:

```
is_settlement = TRUE
        ↓
sub_status should = Settled
```

And:

```
is_waivered = TRUE
        ↓
waiver_amount > 0
```

These are excellent data-quality checks for your project.

---

15. Handle JSON columns

You have many JSON fields:

```
repayment_schedule
transactions
partial_transactions
emi_dates
logs
email_logs
audit_logs
third_party_data
disbursement_details
edited_loan_details
follow_up
docs_information
bank_statement_key
rejection_reason
nbfc
deviation_details
waiver_details
settlement_details
payment_gateway_orders
disbursal_payment_orders
```

Don't flatten everything into the main Silver loan table.

I'd split them.

Silver loan table

Keep core loan-level attributes:

```
loan_id
user_id
amount
status
loan_type
interest_rate
tenure_months
total_payable
total_amount_paid
outstanding_amount
created_at
updated_at
...
```

Separate Silver tables

```
silver.loan_transactions
silver.repayment_schedule
silver.loan_audit_logs
silver.loan_call_logs
silver.loan_rejection_details
silver.loan_disbursement_details
```

This will give you a much better data model.

---

16. Parse `rejection_reason`

Currently it's JSON like:

JSON

```
{
  "code": "REJ123",
  "reason": "Low Credit Score",
  "severity": "High"
}
```

Instead of keeping only the JSON string, extract:

```
rejection_code
rejection_reason
rejection_severity
```

This will make your future:

> Top 10 rejection reasons

analysis extremely easy.

---

17. Parse `third_party_data`

You have:

```json
{
    "bureau_score": 720,
    "income_verified": true,
    "employment_verified": true
}
```

Create:

```
bureau_score
income_verified
employment_verified
```

Then you can build risk analytics directly from Silver.

---

18. Parse NBFC information

Current:

```json
{
    "name": "NBFC_A",
    "application_id": "NBFC123456"
}
```

Extract:

```
nbfc_name
nbfc_application_id
```

---

19. Data quality score

I'd recommend adding a simple quality framework.

For every record, calculate:

```
dq_status
dq_error_count
```

For example:

SQL

```sql
CASE
    WHEN amount <= 0 THEN 1
    WHEN total_payable < amount THEN 1
    WHEN status = 'Disbursed'
         AND disbursed_at IS NULL THEN 1
    WHEN updated_at < created_at THEN 1
    ELSE 0
END
```

Then:

```
dq_status = PASS / FAIL
```

This is excellent for demonstrating real-world DE skills.

---

20. Add audit columns

Your Silver table should have pipeline metadata such as:

```
_ingested_at
_processed_at
_source_system
_pipeline_run_id
_record_hash
```

For example:

Python

Run

```python
from pyspark.sql.functions import current_timestamp, lit

df = (
    df
```

```
    .withColumn("_processed_at", current_timestamp())
    .withColumn("_source_system", lit("AWS_RDS_POSTGRES"))
)
```

Recommended Silver schema

I wouldn't keep all 96 columns in one giant Silver table.

I'd create:

`silver.loans`

```
loan_id
user_id
application_number
amount
status
loan_type
reason

fee
fee_percentage
interest_rate
interest_rate_after_due_date
tenure_months
total_payable
emi_amount

payment_status
paid
total_amount_paid
refund_amount
waiver_amount
total_penalty_amount
bounce_amount

outstanding_amount
net_amount_paid

collection_status
after_disbursal_status
overdue_substatus

is_overdue
is_npa
is_high_dpd
is_90_plus_dpd

category
```

```
decile
is_risky_customer

source
applied_via
platform
policy

officer
assigned_to
collection_agency

created_at
updated_at
disbursed_at
due_date
closed_at
rejected_at
marked_npa_at

dq_status
dq_error_count
_processed_at
_source_system
```

silver.loan_transactions

```
transaction_id
loan_id
amount
transaction_type
status
transaction_timestamp
```

silver.repayment_schedule

```
loan_id
emi_no
emi_amount
emi_status
emi_date
```

silver.loan_rejection

```
loan_id
rejection_code
rejection_reason
severity
```

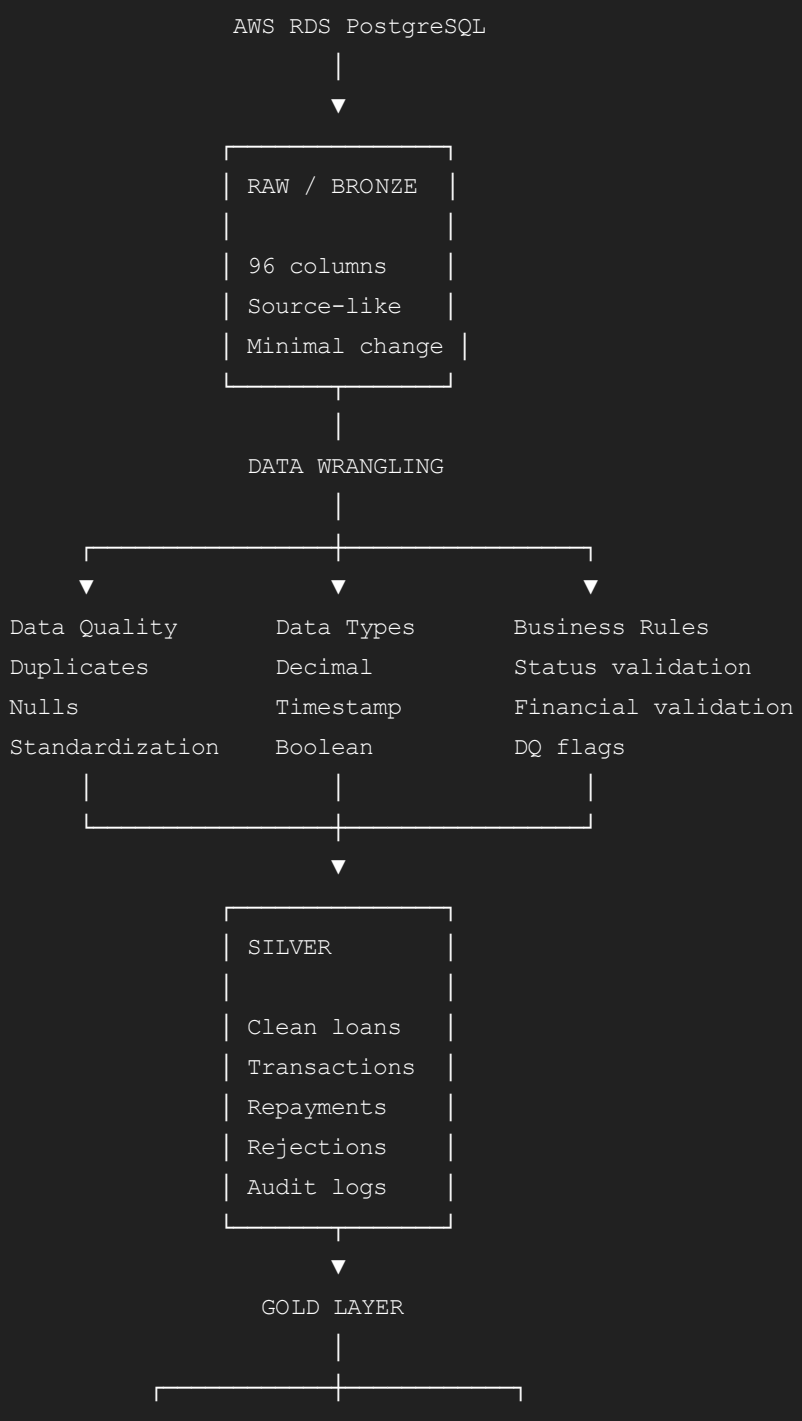
```
rejected_by
```

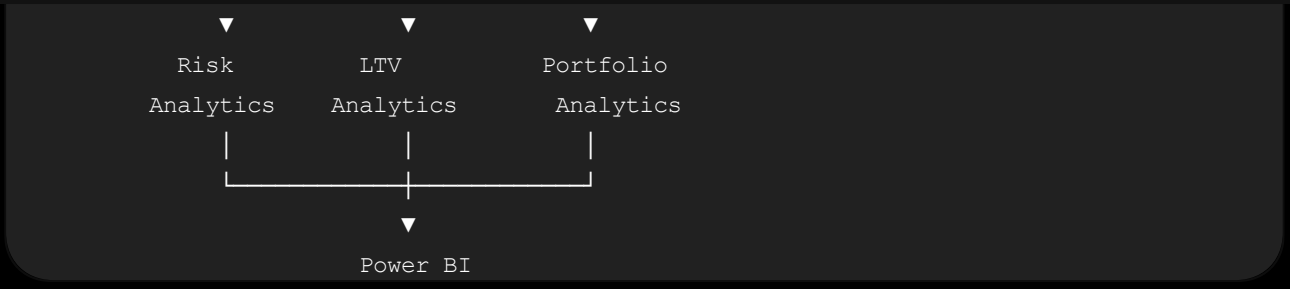
`silver.loan_audit_logs`

```
loan_id
action
performed_by
timestamp
```

Your final architecture

For your project, I'd build it like this:





The biggest principle: Raw should preserve what came from PostgreSQL; **Silver should make the data trustworthy and usable**; Gold should contain business-specific aggregations and KPIs.

For your project, I would **not do heavy aggregation in Silver**. Keep Silver at the loan/transaction/repayment grain, and leave things like monthly disbursement, LTV, default rate, approval rate, and CEO-level KPIs for Gold.